

# ADJv3

## Acta de la 1ª Reunión de comunicaciones

Javier Ribal del Río

2026-10-09

### Asistentes

- Javier Ribal del Río (SW-FW)
- Jorge Canut (FW)
- Víctor López (FW)
- Aleksandr Globa (FW)

## Tabla de contenidos

|       |                                                                                                                       |   |
|-------|-----------------------------------------------------------------------------------------------------------------------|---|
| 1     | Contextualización                                                                                                     | 1 |
| 2     | Modificación de la estructura de los <b>measurements</b> y modificación de los <b>id</b> de los <b>packets/orders</b> | 2 |
| 3     | Incorporación de las <b>protections</b> al <b>ADJ</b>                                                                 | 2 |
| 3.1   | Estructura del paquete                                                                                                | 2 |
| 3.2   | Tipos de <b>protection</b> definidos en <b>general_info.json</b>                                                      | 2 |
| 3.2.1 | Ejemplos de tipos de <b>protection</b>                                                                                | 3 |
| 3.3   | Definición de <b>protections</b> en los <b>measurements</b>                                                           | 3 |
| 3.4   | Id de las protecciones                                                                                                | 4 |
| 4     | Refactor de los <b>messages</b>                                                                                       | 5 |
| 4.1   | Estructura del <b>message</b>                                                                                         | 5 |
| 4.1.1 | Implementación en <b>general_info.json</b>                                                                            | 5 |
| 5     | Adición de la <b>MAC</b> a la información propia de cada placa                                                        | 5 |
| 6     | Puntos a tratar en la próxima reunión                                                                                 | 5 |
| 7     | Otros apuntes de la reunión                                                                                           | 6 |

## 1 Contextualización

El presente documento tiene como objetivo servir como especificación técnica para la implementación del **ADJv3**, partiendo de la base del **ADJv2** definido en el **README.md** del repositorio. El cambio más significativo se corresponde con la separación en forma y concepto de los **messages** y las **protections**. En el pasado, un **message** se correspondía con la información que enviaban las placas al vehículo, y las **protections** eran una implementación de los **messages**; es decir, una **protection** era un **message** con un tipo determinado. Asimismo, el documento también se corresponde con el acta de la primera reunión de comunicaciones.

## 2 Modificación de la estructura de los measurements y modificación de los id de los packets/orders

Con el objetivo de dar soporte a las **protections** para cada **measurement**, el campo **id** pasa a ser un identificador único numérico común entre **packets** y **orders**. El rango de este identificador común es  $[0, 8.191] = [0, 2^{13} - 1]$ <sup>1</sup>. Sin embargo, los **id** comprendidos en  $[0, 511]$  se corresponden con los **ID** especiales: no asignados a **packets**, **orders** ni **protections**, o reservados para otros fines.

Sin embargo, no podemos prescindir del **id** textual previamente definido, puesto que es empleado por la generación de código y no puede ser sustituido por el **id** numérico<sup>2</sup>. Por ello, definimos un nuevo campo denominado **alias** que se corresponde con el antiguo campo **id**<sup>3</sup>.

## 3 Incorporación de las protections al ADJ

Las **protections** se desvinculan de la implementación de los **messages**, por lo que es necesario definir una nueva estructura para el paquete a enviar.

### 3.1 Estructura del paquete

El paquete de una **protection** tiene longitud casi fija: una cabecera de 48 bits (**id** más **timestamp**) seguida del valor que ha disparado la protección, cuyo tamaño depende del tipo del **measurement** vigilado.

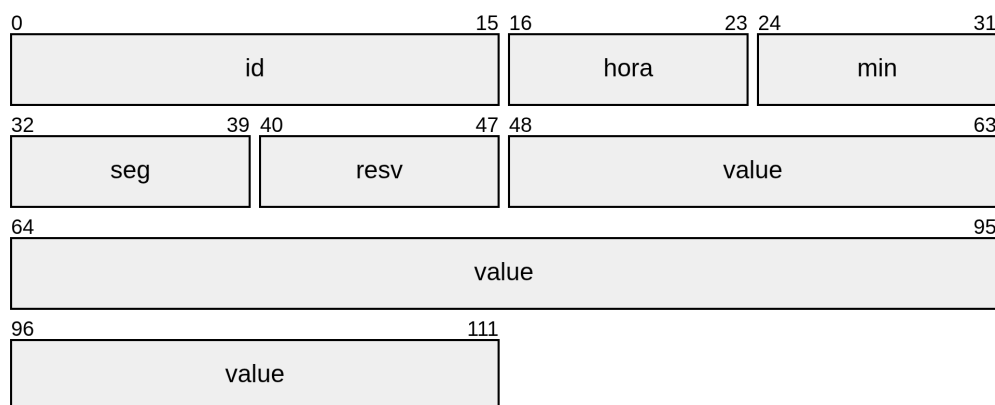


Figure 1: Estructura del paquete de una protección. Las casillas van numeradas por su offset en bits. El campo **value** ocupa el mismo tamaño que el **measurement** asociado (de 8 a 64 bits); aquí se dibuja con 64.

- **id** (bits 0–15): identificador numérico de la protección.
- **timestamp** (bits 16–47): instante en que salta la protección, con **hora**, **min** y **seg** en 8 bits cada uno; los últimos 8 bits quedan reservados para alineación.
- **value** (bits 48 en adelante): valor que ha excedido el umbral. Ocupa exactamente el mismo tamaño que el **measurement** que vigila la protección (de 8 a 64 bits según su **type**), de modo que el receptor lo decodifica con el mismo codificador que ese **measurement**.

### 3.2 Tipos de protection definidos en general\_info.json

Los tipos de **protection** que pueden saltar se declaran en **general\_info.json**, en un objeto **protectionTypes** que asocia el nombre de cada protección a su definición:

```
{
  "protectionTypes": {
    "<protection_name>": {
      "range":    booleano?,
```

<sup>1</sup>Podríamos definirlo como un `int13`

<sup>2</sup>Esto se debe a que actualmente la generación de código utiliza el **id** del **measurement** para generar su variable asociada

<sup>3</sup>Esta decisión de mantener un campo legado maximiza la compatibilidad con la **logging-view**



```

    "text":      "<texto>",
    "textTime": "<texto>?"
  }
}
}

```

- **range**: booleano opcional, **false** por defecto. Indica cuántos límites tiene la protección: **false** para las de un solo valor (**Above**, **Below**, **Equal**...) y **true** para las de intervalo (**Range**), en cuyo caso los dos límites se escriben con notación de intervalo, p. ej. [233, 333].
- **text**: plantilla del mensaje que viaja en el campo **message** del paquete. Los {} se sustituyen por los límites definidos para la protección (un único valor, o el intervalo [min, max] si **range** es **true**) y el %, por el valor que los ha excedido.
- **textTime**: plantilla alternativa para las protecciones de acumulación temporal (**time\_accumulation**), que saltan cuando el valor se mantiene fuera de rango durante un intervalo dado. Se usa en lugar de **text** y el % se sustituye por ese tiempo máximo, no por el valor.

### 3.2.1 Ejemplos de tipos de protection

- Above
- Below
- Equal
- NotEqual <sup>4</sup>
- Range

## 3.3 Definición de protections en los measurements

Cada **measurement** de <PLACA>\_measurements.json incorpora los cambios anteriores (el **id** pasa a ser numérico, de rango [0, 8.191], y el identificador textual antiguo se conserva en **alias**) y, además, una lista opcional **protections** con las protecciones que vigilan esa medida:

```

{
  "id":          número,
  "alias":       "<texto>",
  "name":       "<texto>",
  "type":       "<tipo>",
  "podUnits":   "<nombre de unidad>?",
  "displayUnits": "<nombre de unidad>?",
  "enumValues": ["<texto>"]?,
  "protections": [
    {
      "type": "<tipo de protección>",
      "value": [número] | [número, número],
      "fault": booleano?,
      "time": "<número><unidad>?"
    }
  ]?
}

```

**protections** *array* limitado a 7 protecciones.

- **type**: uno de los tipos declarados en **protectionTypes** (**Above**, **Below**, **Equal**, **NotEqual**, **Range**).
- **value**: lista con el o los límites de la protección. Un elemento para las de un solo valor y dos, [min, max], para **Range**.
- **fault**: booleano opcional, **true** por defecto. Fija la severidad con la que se emite la protección: **true** la envía como *fault* y **false** como *warning*.
- **time**: opcional, solo para las protecciones de acumulación temporal. Cadena con valor y unidad ("500ms", "3s"...) que indica cuánto tiempo debe mantenerse la medida fuera de rango antes de que la protección salte, por defecto 0.

<sup>4</sup>La protección se encuentra definida en la ST-LIB, por el momento no se nos ha ocurrido su propósito

### 3.4 Id de las protecciones

#### ¿Qué es *bit-packing*?

Es la técnica de meter varios campos independientes dentro de un mismo entero, cada uno ocupando solo los bits que necesita, en vez de reservar un byte o un campo propio. Aquí el `uint16` del `id` empaqueta dos campos: `pos` (3 bits) e `id_measurement` (13 bits). Se construye desplazando y combinando con operadores a nivel de bit — (`pos << 13`) | `id_measurement` — y se deshace igual, en el otro sentido: `pos = id >> 13` e `id_measurement = id & 0x1FFF`. La ventaja es que no se desperdicia ni un bit; el coste es que, si un consumidor no sabe que el campo va empaquetado, lo interpreta mal sin que nada se lo avise.

Esta codificación no es exclusiva del paquete de la `protection`: como `packets`, `orders` y `protections` comparten el mismo campo `id` de 16 bits, se aplica al `id` de **cualquier** paquete del protocolo. Al recibir un `id`, lo primero que se mira son sus 3 bits altos (`pos`): si valen 000, el resto son los 13 bits del `id` normal de un `packet`, `order` o `measurement`; si no, indican qué `protection` (1–7) del `measurement` de esos 13 bits ha saltado:

$$\text{id} = (\text{pos} \ll 13) | \text{id\_measurement}$$

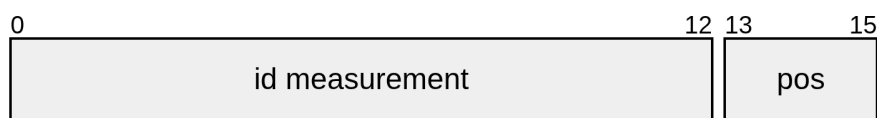


Figure 2: Id de 16 bits compartido por `packets`, `orders` y `protections`: 3 bits de posición (000 = `id` normal) + 13 bits del `id` de `measurement`/`packet`/`order`.

| pos | Significado                                                                                 |
|-----|---------------------------------------------------------------------------------------------|
| 000 | <code>id</code> normal: <code>packet</code> , <code>order</code> o <code>measurement</code> |
| 001 | <code>protections</code> [0]                                                                |
| 010 | <code>protections</code> [1]                                                                |
| 011 | <code>protections</code> [2]                                                                |
| 100 | <code>protections</code> [3]                                                                |
| 101 | <code>protections</code> [4]                                                                |
| 110 | <code>protections</code> [5]                                                                |
| 111 | <code>protections</code> [6]                                                                |

Con `pos` de 1 a 7 caben exactamente las 7 `protections` por `measurement` fijadas antes; `pos` = 0 deja el `id` libre para su uso normal. Como el resto del protocolo, este `id` compuesto viaja como un único `uint16` en *little-endian* <sup>5</sup>.

<sup>5</sup>`pos` no ocupa un byte aparte: son los 3 bits más altos de ese mismo `uint16`, así que quedan repartidos en el byte alto junto con los 5 bits más significativos de `id_measurement`.

## 4 Refactor de los messages

### 4.1 Estructura del message

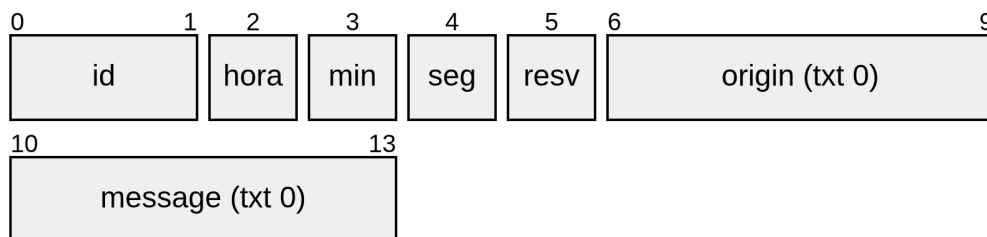


Figure 3: Estructura del nuevo message. origin (máx. 48 bytes) y message (máx. 320 bytes) son cadenas terminadas en 0, de longitud variable; aquí se dibujan truncadas a 4 bytes.

- **id** (2 bytes): tipo de mensaje (uint16). Reutiliza `message_ids` de `general_info.json` (ver más abajo): 10 = `fault`, 11 = `warning`, 12 = `info` y, como tipo nuevo, 13 = `panic`<sup>6</sup>.
- **timestamp** (4 bytes): el mismo formato corto que en las `protections` — `hora`, `min` y `seg` en un byte cada uno, con el cuarto reservado.
- **origin** (hasta 48 bytes, cadena terminada en 0): de dónde sale el mensaje. Actualmente es la ruta absoluta del fichero de origen <sup>7</sup>.
- **message** (hasta 320 bytes, cadena terminada en 0): contenido del mensaje.

`origin` y `message` llevan tamaño máximo pero longitud variable: al llegar al byte 0 se entiende que la cadena ha terminado.

#### 4.1.1 Implementación en `general_info.json`

Reutilizamos los `message_ids` definidos en `general_info.json`, que por el momento estaban en desuso:

```
{
  "message_ids": {
    "fault": 10,
    "warning": 11,
    "info": 12,
    "panic": 13
  }
}
```

## 5 Adición de la MAC a la información propia de cada placa

En vista de lo acontecido en la EHW en H11<sup>8</sup>, se va a añadir un campo `mac` que se corresponde con la MAC de cada placa con el objetivo de que se use en la generación de código y sea validado por el `adj-validator`.

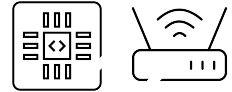
## 6 Puntos a tratar en la próxima reunión

1. Ratificar implementación de las `protections` y refactor de los `messages`
2. Valorar la utilidad del módulo de `units` y considerar su desimplementación
3. Asignar rango dentro de los `id` especiales para la BLCU
4. Debater sobre la verificación del `commit` del ADJ al iniciar la conexión por TCP
5. Debater sobre el `custom keep-alive`

<sup>6</sup>Ya estaba definido en la ST-LIB y era enmascarado como un mensaje de `fault`, ahora queda incluido en el ADJ

<sup>7</sup>Sería mejor que fuese la ruta relativa desde la raíz del repositorio de la placa, no la ruta absoluta del sistema en el que se compiló.

<sup>8</sup>Hubo MAC *curiosas* como 67:67:67:67:67:67 que causaban problemas y horas de *debugging*



6. Valoración sobre la inclusión de los **arrays** de **measurements**. Víctor vendrá con una propuesta de implementación

## 7 Otros apuntes de la reunión

Por la parte de FW, Víctor y Jorge Canut se muestran conformes con la implementación del refactor en la ST-LIB. Por otra parte, Javier Ribal comenta que al rehacerse el **backend** desde cero no ve ningún impedimento en aplicar estos cambios.

Primera reunión de Aleksandr Globa, que se incorpora al subsistema de Firmware.

